PROJECT PLAN

Self-Correcting, Execution-Aware LLM Framework
for Database Query and API Generation

Version 1.0 | March 2026

1. Project Overview

This project builds a self-correcting, execution-aware LLM-powered system that converts natural language instructions into valid SQL queries and production-ready backend APIs. Unlike conventional one-shot LLM tools, this system integrates directly with a live MySQL database, validates every generated output through actual execution, and iteratively corrects mistakes using real error feedback.

The core innovation is the closed feedback loop: generate → execute → capture error → re-generate. This loop continues until the output is both syntactically valid and semantically correct, dramatically improving reliability over baseline LLM systems.

2. Objectives

- Build an LLM-powered system for automated SQL query generation from natural language.
- Integrate execution feedback for iterative self-correction.
- Ensure all query generation is schema-aware to eliminate hallucinated columns/tables.
- Automate CRUD API creation (FastAPI) directly from validated database operations.
- Deliver query optimization suggestions and human-readable explanations.
- Demonstrate measurable reliability improvement over single-pass LLM approaches.

3. System Architecture

The system is composed of nine discrete modules orchestrated by a central Controller. The high-level data flow is:

User Input → Planner → Schema Retriever → SQL Generator → Validator → Execution Engine → Self-Correction Module → API Generator → Explanation Module → Output

3.1 Module Descriptions

Planner Module

Decomposes the user's natural language request into structured, actionable sub-tasks for downstream modules.

Schema Retrieval Module

Connects to the live database and fetches the current schema (tables, columns, types, relationships) using metadata queries. This ensures the LLM always works with accurate schema context.

SQL Generator Module

Uses the LLM (with schema context) to generate SQL queries. Prompts are engineered to be schema-grounded and purpose-specific.

Validator Module

Performs pre-execution checks for syntax correctness and schema compliance before the query reaches the database.

Execution Module

Runs the validated SQL against the live MySQL database, captures query results, and surfaces execution errors for the correction loop.

Self-Correction Module

The core innovation. Takes execution errors and feeds them back into the LLM with the original intent and schema to produce a revised, improved query. Loops until success or max iterations.

API Generator Module

Translates validated database operations into FastAPI CRUD route definitions, complete with request/response models and DB bindings.

Controller Module

Orchestrates the entire workflow — manages iteration counts, routes signals between modules, enforces retry limits, and handles failure states.

Explanation Module

Generates plain-English summaries of the final SQL query and its results for end-user readability.

4. Implementation Phases

The project is structured into five sequential phases across five weeks, each with clear deliverables and entry/exit criteria.

Phase	Name	Duration	Key Tasks	Deliverable
Phase 1	Setup & Infrastructure	Week 1	DB connection, schema fetching, environment setup	Connected DB + Schema Retriever
Phase 2	Core SQL Engine	Week 2	SQL generation, execution module, error capture	Working SQL Generator
Phase 3	Self-Correction Loop	Week 3	Retry loop, error-fed prompts, validator module	Self-Correcting Query Engine
Phase 4	API Generation	Week 4	CRUD API generation, FastAPI routes, DB binding	Auto-generated REST APIs

Phase	Name	Duration	Key Tasks	Deliverable
Phase 5	UI, Testing & Evaluation	Week 5	Streamlit UI, evaluation metrics, optimization	Full Working System

Phase 1 — Setup & Infrastructure (Week 1)

Establish the project environment and database connectivity. This phase produces the foundational layer everything else depends on.

- Configure Python virtual environment and install core dependencies (FastAPI, SQLAlchemy, OpenAI SDK).
- Establish MySQL database connection with environment-based credential management.
- Implement the Schema Retrieval Module using INFORMATION_SCHEMA queries.
- Write unit tests to validate schema retrieval accuracy.
- Set up project structure, logging, and configuration management.

Phase 2 — Core SQL Engine (Week 2)

Build the SQL generation and execution backbone. At the end of this phase, the system can generate and run queries — without self-correction yet.

- Implement the Planner Module with prompt templates for task decomposition.
- Build the SQL Generator Module with schema-injected prompts.
- Implement the Validator Module for syntax and schema pre-checks.
- Build the Execution Module with result capture and error surfacing.
- Create integration tests covering SQL generation → execution flow.

Phase 3 — Self-Correction Loop (Week 3)

Introduce the system's defining capability: iterative self-correction using execution feedback.

- Implement the Self-Correction Module with error-context prompt construction.
- Build the Controller Module to manage retry loops with a configurable max-iteration cap.
- Engineer error-aware prompt templates that include the original query, the error message, and the schema.
- Benchmark correction success rate vs. one-shot baseline.
- Add correction iteration count logging for evaluation metrics.

Phase 4 — API Generation (Week 4)

Extend the system to automatically produce production-ready CRUD APIs from validated database operations.

- Implement the API Generator Module that maps validated queries to FastAPI route definitions.
- Generate Pydantic request/response models for each CRUD operation.
- Bind generated APIs to the MySQL database via SQLAlchemy ORM.
- Auto-generate OpenAPI documentation for all endpoints.
- Test generated APIs with sample payloads across all CRUD operations.

Phase 5 — UI, Testing & Evaluation (Week 5)

Package the system for end-users and conduct comprehensive evaluation.

- Build Streamlit frontend with query input, result display, and API viewer.
- Run the full evaluation suite against all defined metrics (success rate, latency, accuracy, iteration count).
- Conduct end-to-end testing with a representative set of 20+ natural language queries.
- Document limitations and known edge cases.
- Prepare final demo and project report.

5. Technology Stack

Category	Tool / Technology	Purpose
Language	Python 3.10+	Core development language
Backend	FastAPI	REST API framework for generated CRUD endpoints
Database	MySQL	Primary relational database for query execution
LLM	OpenAI API / Local LLM	Natural language to SQL/API generation
Orchestration	LangChain (optional)	Prompt chaining and workflow management
UI	Streamlit	Lightweight frontend for user interaction
Testing	pytest	Unit and integration test suite

6. Evaluation Metrics

System performance will be measured across the following dimensions:

Metric	Definition	Target
Query Success Rate	% of queries that execute without error	> 85%
Execution Accuracy	Correct result vs. expected output	> 80%
Correction Iterations	Average loops before successful query	< 3
Latency	End-to-end response time	< 10s per query
API Correctness	Generated API routes pass test suite	> 90%

7. Limitations & Risks

LLM Quality Dependency

Output quality is bounded by the underlying model. Weak models will require more correction iterations or may not converge.

Latency Overhead

The correction loop introduces additional API calls per query, increasing end-to-end latency compared to single-pass systems.

Complex Query Handling

Highly nested subqueries, recursive CTEs, and multi-join queries may exceed the system's correction capability within the iteration limit.

Security Considerations

The current implementation does not include authentication, rate limiting, or SQL injection safeguards — planned for future work.

8. Future Work

- Multi-database support (PostgreSQL, SQLite, MongoDB).
- Authentication and role-based access control layers.
- Advanced query optimization using EXPLAIN analysis.
- Extension to full-stack code generation (frontend + backend).
- Fine-tuned domain-specific LLM for improved schema comprehension.

PROJECT ONE-PAGER

Self-Correcting, Execution-Aware LLM Framework
for Database Query and API Generation

THE PROBLEM

LLM-based query tools hallucinate table names, generate unexecutable SQL, and lack any validation feedback. One-shot systems fail in real-world production environments.

THE SOLUTION

A closed-loop, schema-aware system that generates SQL, executes it, captures errors, and automatically refines queries until they succeed — then generates production-ready CRUD APIs.

KEY DIFFERENTIATORS

- Execution-based validation (not just syntax)
- Iterative self-correction via error feedback
- Schema-aware generation (no hallucinated columns)
- Automated CRUD API generation
- Human-readable query explanations

HOW IT WORKS

1. User submits natural language query
2. System retrieves live DB schema
3. LLM generates SQL / API code
4. Validator checks syntax & schema
5. Execution engine runs query on DB
6. On error → feed back to LLM for correction
7. On success → generate FastAPI routes
8. Return results + explanation to user

TECH SNAPSHOT

Python · FastAPI · MySQL · OpenAI API · LangChain · Streamlit

TIMELINE

5 Phases · 5 Weeks · MVP to Production-Ready

EXPECTED OUTCOME

A reliable, production-grade system that converts plain English into validated SQL and REST APIs — dramatically reducing developer effort for routine database operations.